

Lab4 AST2IRP

0. Questions

Q2

数组初始化：见 10. `NewArray`。

数组存取：见 5. `ArrayExp`，得到地址表达式。

数组长度：见 9. `Length`。

Q3

class method的参数列表在最开头有this指针参数。

class method中的 `this`：见 8. `This`。

如何记录不同 class 的变量和方法：把字段区和方法槽区放进同一对象内存。字段偏移保存在 `var_pos_map`（键为 类名^字段名），方法偏移保存在 `method_pos_map`（键为方法名，确保同名方法跨类槽位一致）。对象总大小由字段末端和方法槽末端共同决定，分配时一次性 `malloc`。这使得字段访问可按固定偏移取值，方法调用可按固定槽位取函数指针。

如何处理多态：多态通过“构造时写槽位 + 调用时按槽位间接跳转”实现。new 对象时，先根据当前类和继承链解析每个方法最终实现类，再把对应函数标签写入对象方法槽。调用 `obj.m(...)` 时，不是直接按静态类名调用，而是先从 `obj` + 方法槽偏移取函数指针，再执行间接调用，并把 `obj` 作为第一个实参传入。因此同一槽位在不同对象实例里可指向不同实现，从而实现动态分派。

Class初始化：new class 时分配统一对象大小，并初始化方法槽为最终实现函数标签。

访问类变量：先确定对象表达式的实际类，再沿继承链解析字段归属类，最后用 对象基址 + 字段偏移 做内存读写。

访问类方法：先求对象值，再按方法槽偏移取函数指针，参数列表首位插入对象指针，然后发起调用。

1. Program

先翻译main，再逐个翻译class中的method。

2. append_vardecls_init

传入 `vdI` 列表，`method_var_table` 和 `temp_map`，把每个 `VarDecl` 都尝试转成初始化 IR。对 `vd->init` 使用 `holds_alternative`，来判断根据不同的初始化分别处理：`monostate` 对指针类型补 `dst = 0;`

`IntExp*` 生成 `Move(dst, Const(v));` `vector<IntExp*>` 先 `malloc` 分配 `(n+1)*int_len` 再把长度写到数组头部，再逐元素写到偏移 `(i+1)*int_len`。`vdI` 生成的语句全部append到 `sl` 中。保证初始化统一在方法体语句前执行，符合先声明初始化、后执行语句的语义。

3. MethodDecl

翻译类方法，生成 `"cClass^method"` 的 `FuncDecl`。先创建该方法的 `method_temp_map` 和 `method_var_table`。然后翻译 `FuncDecl` 的参数 `temp` 列表。先放 `this`，再按顺序放入 `formal` 对应的 `temp`。

然后新建一个 `vector<tree::Stm*> *sl`。调用 `append_vardecls_init` 来把变量初始化语句先放入。然后逐条访问 `node->sl` 并把结果转换为 `tree::Stm*` 插入到 `sl` 中。

最后把返回值用 `ast_type_to_tree_type`，然后组合并返回 `FuncDecl`。

4. CallStm

如果 `node->obj == nullptr`，说明为外部调用，逐个翻译参数，统一转成 `Exp` 形态，便于作为调用参数。然后直接构造构造 `ExtCall`。

为对象方法调用的话，先翻译对象表达式 `obj`，然后从类表中找 `method_pos_map`。然后合成一个 `tree::Mem`。把对象指针作为第一个参数插入，再追加显式参数。最后构造一个 `Call`，返回 `type` 默认为 `INT`，转化为 `ExpStm`（不要返回值，保留副作用）。

5. ArrayExp

先分别处理 `array` 和下标，把副作用存到 `vector<tree::Stm*>` 中。调用 `checked_array_index_addr`，检查越界，合法时返回地址表达式。然后合成一个 `Esep`，先执行副作用语句，再返回地址值。最后封装为 `Tr_ex`。

6. CallExp

和 `CallStm` 类似，但是有返回值。用 `semant_to_tree_type` 转换出返回类型，将结果转化为 `Tr_ex` 类型返回。

7. ClassVar

先从 `Name_Maps` 中获取变量的类，然后用 `class_table->get_var_pos`，得到变量在类里的偏移量。然后得到 `Type` 和 `addr`，包装返回 `Tr_ex`。

8. This

从当前 `method_var_table` 中查找 `this` 对应的 `temp`，返回 `Tr_ex`。前面在方法变量表中先存放了 `_^this^_` 对应的 `temp`。

9. Length

拼接为一个 `Eseq`，先执行语句部分 `Move(len_t, Mem(INT, arr))`，即从 `arr` 指向的首地址读出长度。再返回表达式部分 `len_t`。然后打包成 `Tr_ex`。

10. NewArray

先翻译数组长度表达式，把 `size` 子表达式变成 `IR`。然后计算总分配字节数，`bytes = (size + 1) * int_len`，这里 `+1` 是因为数组头部额外存了长度。调用 `malloc` 分配内存，把返回指针写到临时变量 `base`。把长度写入数组头，执行 `Mem(base) = size`。最后用 `Eseq` 返回数组指针，返回一个指针类型表达式：先做 `sl` 里的初始化，再把 `base` 当值返回。

11. NewObject

准备语义表和对象大小，拿 `Name_Maps`，并通过 `compute_unified_object_size` 计算统一对象大小。这个大小覆盖了字段区和方法槽区（统一记录布局）。用 `malloc` 分配对象内存并写到 `base`。

遍历 `class_table` 里的所有方法槽位 `method_pos_map`。对每个方法名，调用 `resolve_method_impl` 找当前类 `cls` 在继承链上真正的实现类 `impl_class`。如果确实存在该方法，就把函数标签 `impl_class^method_name` 写入对象的对应槽位地址 `base + off`。后续对象调用就能按槽位取函数地址完成分派。

最后用 `Eseq` 返回对象指针，同样是先执行初始化语句，再返回 `base`。